Distributed Database concept

What is Distributed Database?

- Distributed Database comprises:
  1. Autonomous sites
     - mean each site has a DBMS, a database (each site has to be able to work on it own?)
  2. The sites are interconnected (via computer networks)
  3. There are local applications on each site (run on each site and only use resource on each site → if remove network, need to still be able to work)
  4. There are global applications which use resources(processing power, data, ..) from more than 1 site.
     a. very low percentage

  Among this 4 factors, local appkication is the main factor

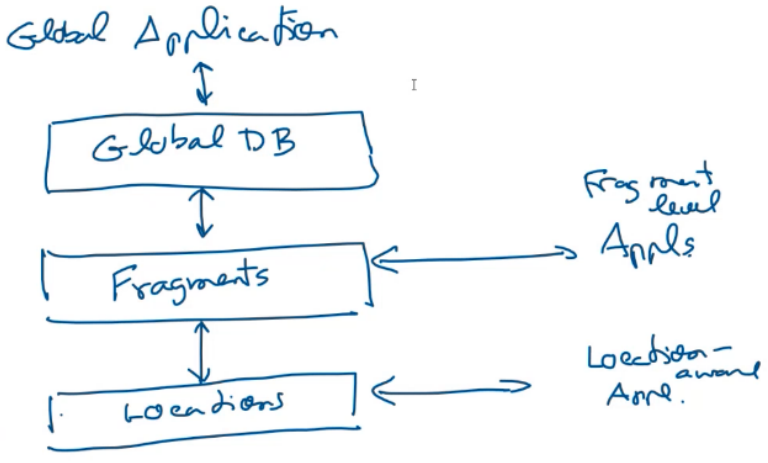
  - lot of people mistunderstand that if we have global app → we should go distributed database
  - if you have more local application → you can go distributed config.
  - example, bank, in banking env, we normally not use distributed → because we have to refer to data in other branch all the time
  - in the past, there one bank went distributed database
    - each branch has it own small computer center. → keep customer info at each branch
    - quite successful (30 years ago)
      - 30 years ago → communication network → very bad
      - prof ask network engineer → fixing network link → how come your system survive → because more than 80% were local customer

        - people who use service of ladkrabang branch are ladkrabang people and they not mind if network is down. this kind of config might not suitable for nowadays.
            - local application is a key
            - company wiht many factory outside bangkok use this → because there have lot of local applciation runnign
    - like if 70% of data are replicate, they might not really want distributed system → they might want centralized system (share info) (global centralized database)

Distributed DB Architecture (summarize)

- the ultimate idea of distributed db is to look at distributed site as one big site → to have logical perception of centralized database
- like don't have to know whre data is

- like select * from supplier (don't have to know where supplier located at)
- in some system select * from s(at newyork)

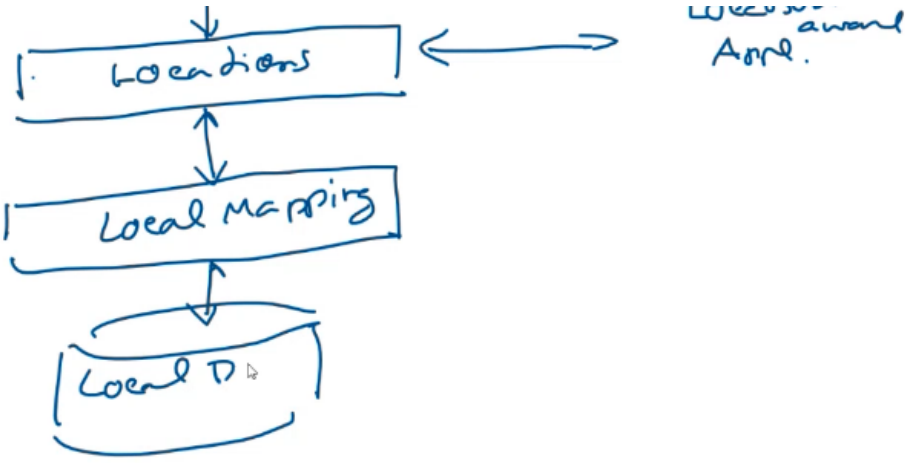


- global application work on global database
- under this global databse → we have fragment
- we may fragment global database

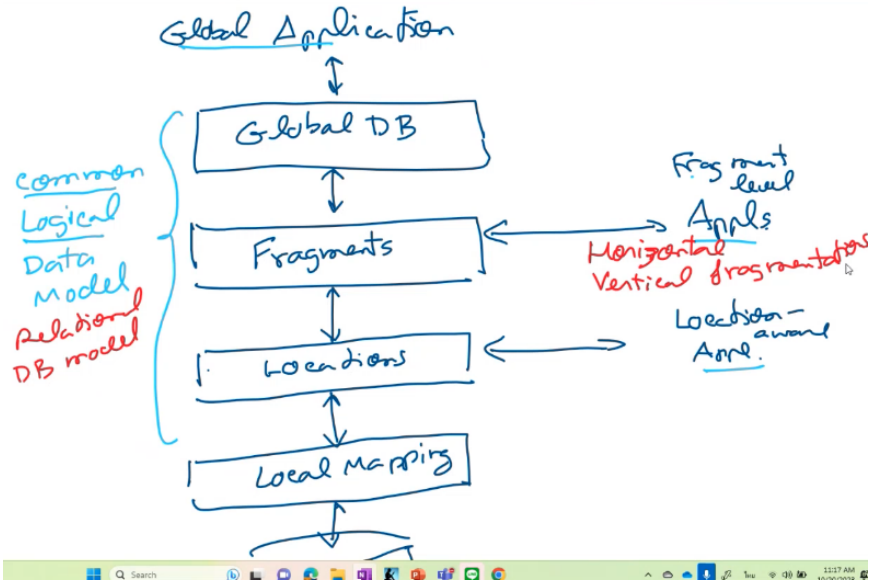
    - like we may have many fragment → but we dont have to know where they are.
- we have app that work on fragment level (see those fragment)
    - and under fragment, we have location
    - say a north fragment may have copy in chiangmai, lumpoon, phitsanulok,.....
        - fragment maybe replicate on copy in each location
        - we have program that work on location (Location aware app)

we assume that this all 3 level → use same database model (tree,graph, table)

- what if individual site use different database model
- you gonna have local mapping which map upper perception to lower level representation (local dbms)



- thing maybe complicate



- should have common logical data model → relational db model

---

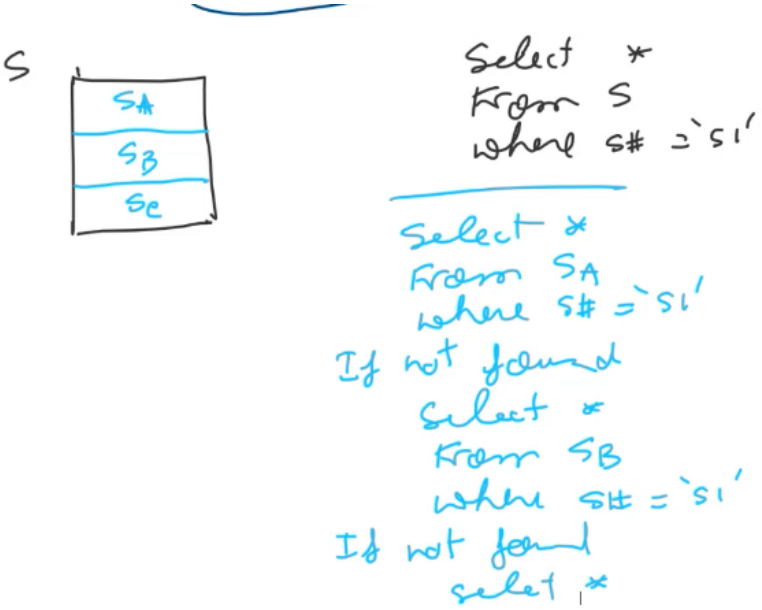
- table s
- suppose you fragment s

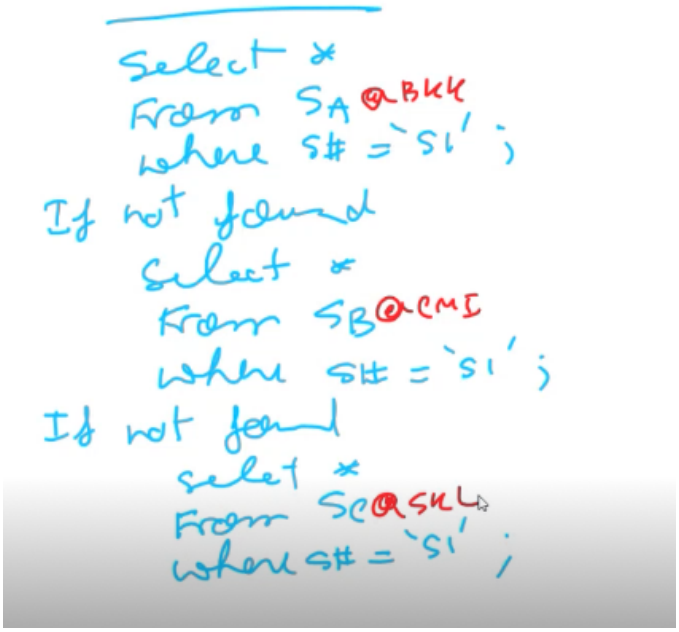
at global level you said

```
select *
from s
when s# = 's1'
# global elvel query
```

- so if you want to find s1 you have to said

  select * from sA where s# = ‘s1’ if not found.....



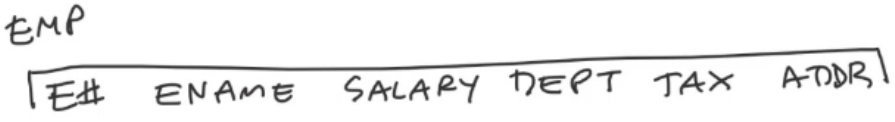


- local mapping translate to other local language??

this is just select, what about isnert, update, delete.

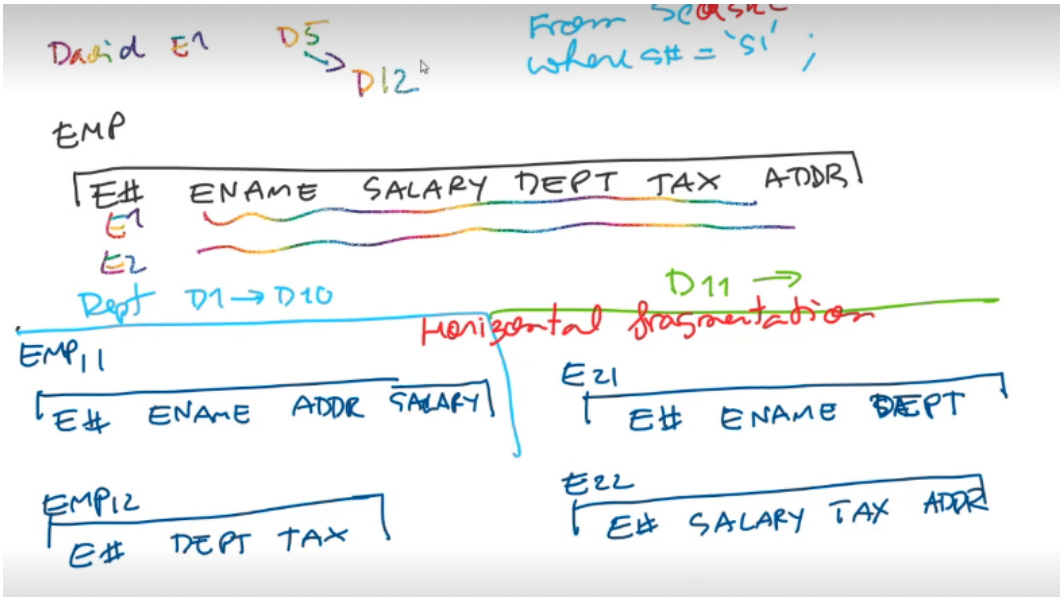
Fragmentation

EMP is global level

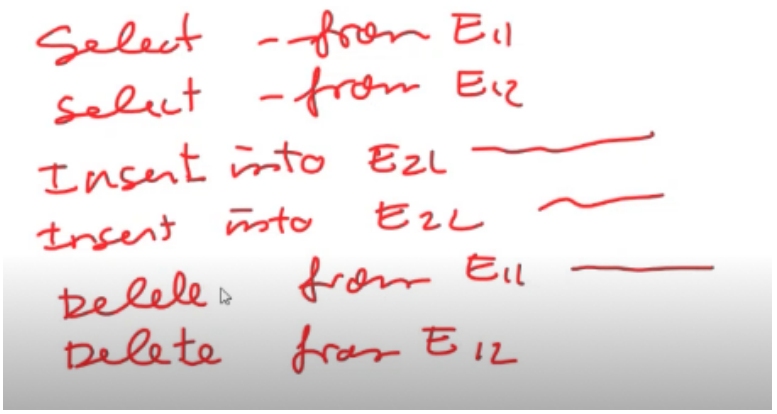


- Horizontal fragmentation → by rows

separate row into 2 fragement based on department



- what if one day, david chagne from department d5 to d12, what happened?
  - at global level is so easy
    - update emp set dept = ‘D12’ where E# = ‘E1’
  - how would you do it at fragment level
    - you have to read from emp11, emp12 → rearrange → insert E21, 22
    - then delete rform emp11, emp12



- whole thing here is 1 transaction (1 distributed transaction) (at fragment level)
- what if you have to take care lcoaiton, and locla mapping too

So, when we study all those topic (concurrency, ...)

- we only consider 1 site
- if have to consider many site
  - global commit, local commit (life become alot harder)

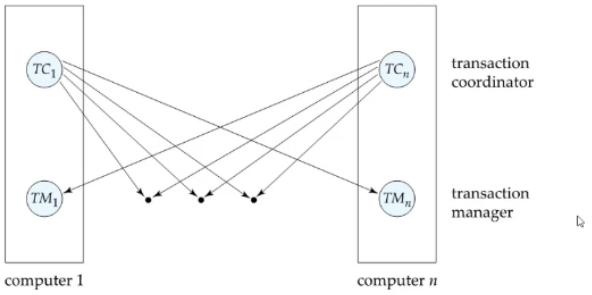
Distributed system → need complex transaction processing

Distributed Transactions

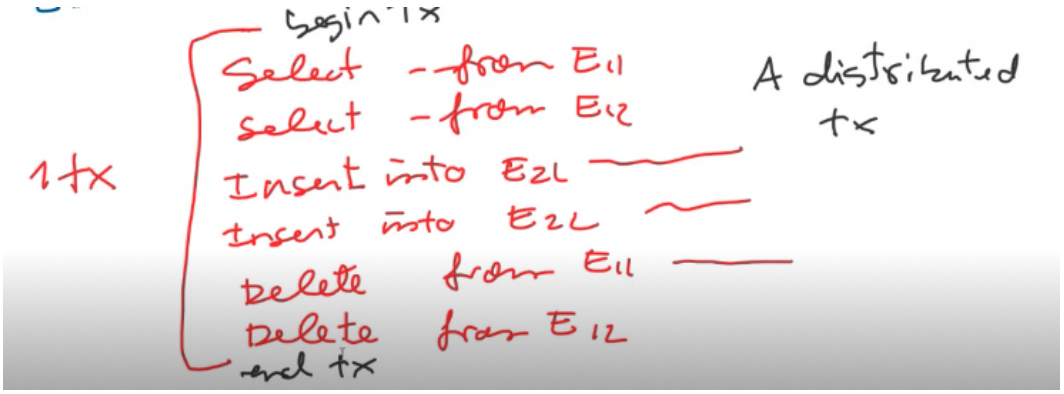
- **Local transactions**
  - Access/update data at only one database
- **Global transactions**
  - Access/update data at more than one database
- Key issue: how to ensure ACID properties for transactions in a system with global transactions spanning multiple database

Distributed Transactions

- Transaction may access data at several sites.
  - Each site has a local **transaction manager**
  - Each site has a **transaction coordinator**
    - Global transactions submitted to any transaction coordinator



- on each site you has to have transaction manager (local) LTM
  - and a global transaction coordinator (transaction coordiator) DTM distributed transaction manager
    - work as coordinate global commit



each sql in global distribuited → you ned to have local commit of like insert, delete

- you also have to have global commit → perform by DTM

we need 2Phase commit protocol (there many protocol, this is the most basic one)

Commit Protocols

- Commit protocols are used to ensure atomicity across sites
  - a transaction which executes at multiple sites must either be committed at all the sites, or aborted at all the sites.
    - cannot have transaction committed at one site and aborted at another
- The *two-phase commit* (2PC) protocol is widely used
- *Three-phase commit* (3PC) protocol avoids some drawbacks of 2PC, but is more complex
- *Consensus protocols* solve a more general problem, but can be used for atomic commit
  - More on these later in the chapter
- The protocols we study all assume **fail-stop** model – failed sites simply stop working, and do not cause any other harm, such as sending incorrect messages to other sites.
  - Protocols that can tolerate some number of malicious sites discussed in bibliographic notes online

- global commit and local commit

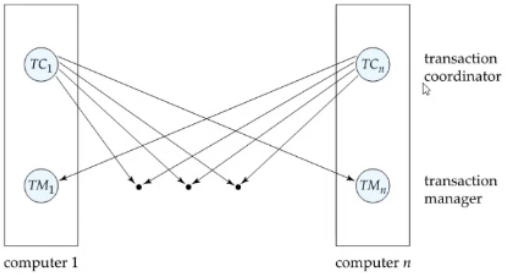
Two Phase Commit Protocol (2PC)

- Execution of the protocol is initiated by the coordinator after the last step of the transaction has been reached.
- The protocol involves all the local sites at which the transaction executed
- Protocol has two phases
- Let T be a transaction initiated at site S_i, and let the transaction coordinator at S_i be C_i

- 2pc, execute of protocol is initited by the coordinator after..... (above)
- **who is the coordinator?**
  - each site must have LTM and DTM, in principle when you initialize a request of transaction at a site
    - the DTM of that site become transaction coordinator (like there no centralized site)

Distributed Transactions

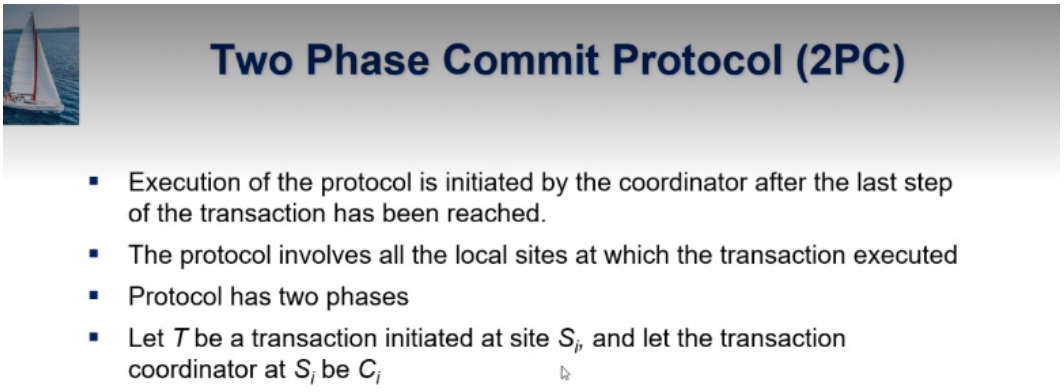
- Each transaction coordinator is responsible for:
  - Starting the execution of transactions that originate at the site.
  - Distributing subtransactions at appropriate sites for execution.
  - Coordinating the termination of each transaction that originates at the site
    - transaction must be committed at all sites or aborted at all sites.
- Each local transaction manager responsible for:
  - Maintaining a log for recovery purposes
  - Coordinating the execution and commit/abort of the transactions executing at that site.



2PC

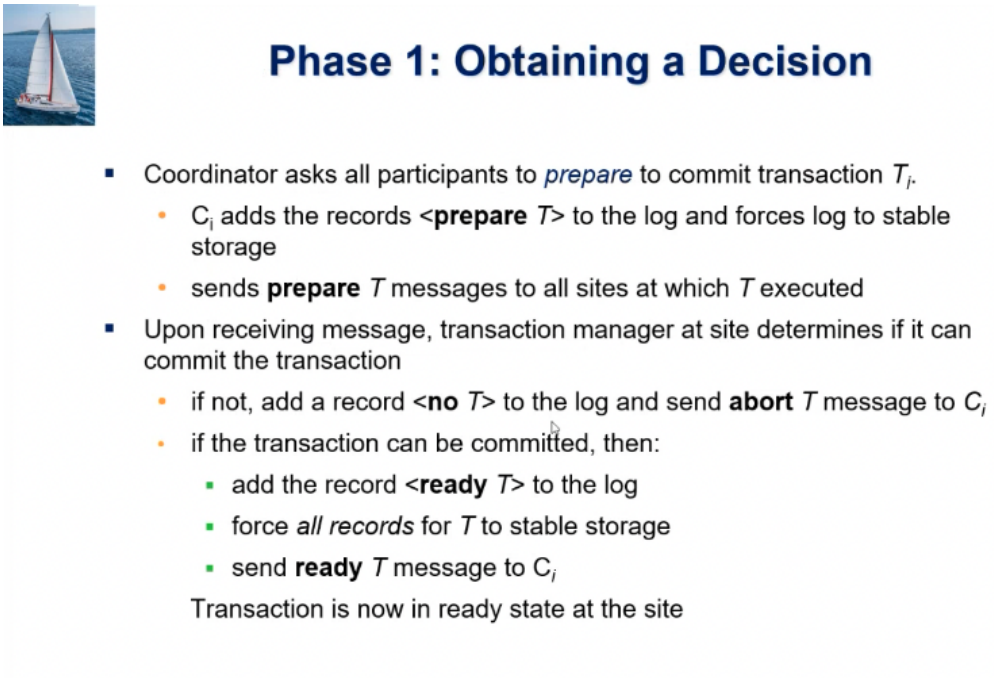
2PC is requried for distributed ACID transactions. (not only apply to relational database) (it apply all?)

- in practice, 2 PC may be expensive (all site must be up together)
- instead of using 2PC, we may consider eventaula ocnsistency. (if possible)
  - commit separately
  - if you use this → might be a sign you need distributed database
  - if you only use 2PC, use alot → maybe a sign you don’t really need distributed database (but centralized one)
  - most no sql nowadays that support data distribution concentrate on read-only application



- Ci is transaction coordinator at site Si

Phase 1: obtaining a Decision



- Coordinator asks all participants to prepare to commit Transation Ti.
  - send prepare, send prepare out
- The participating site receive message, the local site receive message.
  - determine if it can commit, if it can't → local part fail

Basically, the coordinator send prepare, and if all participant send agree, no problem, they all send ready back

- if one send no → the whole will be abort.

Phase 2: Recording the Decision

Phase 2: Recording the Decision

- *T* can be committed of C_i received a **ready** *T* message from all the participating sites: otherwise *T* must be aborted.
- Coordinator adds a decision record, <**commit** *T*> or <a**bort** *T*>, to the log and forces record onto stable storage. Once the record stable storage it is irrevocable (even if failures occur)
- Coordinator sends a message to each participant informing it of the decision (commit or abort)
- Participants take appropriate action locally.

- add commit or abort to the log
- in pracitce, after participant commit or abort, each participant should send acknowledge back to coordinator.
    - because it possible that send prepare, send ready and fail (because immediately fail after sending ready) → this could happen.

Exam

- 5-6 question → all descritive
- first → definition
- the rest → descriptive + some calculation

(query processing, optimization → have some calculation)

- calculator is allow
- note are allow
- cover part after transaction onward → no normalization, no orm.